

## Appendix 2

### THE SIMULATION CODES

This appendix provides the simulation codes for the proposed algorithms, which are performed MATLAB R2017a, windows 7, and 64GB RAM with a classical computer.

#### The Codes for Qubit Gates in Chapter 2

**Code B.1** Calculate  $n$ -qubit basis state  $|j\rangle$  in (2.31)

```
function st = state(j,n)
f=dec2bin(j); % Convert from decimal to binary
[m1,m2]=size(f);
s=strcat(repmat('0',1,n-m2), f);
[m3,m4]=size(s);
a0=[1;0];
a1=[0;1];
for k=1:m3
    kj=1;
    for i=1:m4
        if s(k,i)=='0'
            t=a0;
        else
            t=a1;
        end
        kj=kron(kj,t);
    end
    st(:,k)=kj;
end
end
```

**Code B.2** The implementation of multiple tensor product  $U^{\otimes n}$ 

```

function Y=tenProduct(U,n)
    Y=0;
    if n>0
        Y=U;
        for i=1:n-1
            Y=kron(Y,u);
        end
    end
end

```

**Code B.3** The implementation of the gate  $V_n^i(U)$  in (2.32)

```

function P=Cun(n,i,U)
    [m1,m2]=size(U);
    k=log2(m1);
    I=[1 0;0 1];
    tj=state(i,n); % Calculate |i>
    P=kron(U-tenProduct(I,k), tj*tj')+tenProduct(I, n+k);
end

```

**Code B.4** The implementation of the gate  $C_n^i(U)$  in (2.32)

```

function P=Cup(n,i,U)
    [m1,m2]=size(U);
    k=log2(m1);
    I=[1 0;0 1];
    tj=state(i,n); % Calculate |i>
    P=kron(tj*tj', U-tenProduct(I,k))+tenProduct(I, n+k);
end

```

**The Codes for NASS States in Chapter 3****Code B.5** Converting a 2D image  $A_2$  to a column vector  $A_1$ 

```

function A1=d2tod1(A2)
    [d1,d2]=size(A2);
    A1=zeros(d1*d2,1);
    m=0;
    for i=1:d1
        for j=1:d2
            m=(i-1)*d2+j;
            A1(m)=A2(i,j);
        end
    end

```

## Appendix 2

```
        end
    end
end
```

### Code B.6 Converting a column vector $A1$ to a 2D image $A2$

```
function A2=d1tod2(A1,d1,d2)
A2=zeros(d1,d2);
for i=1:d1
    A2(i,1:d2)=A1(1+(i-1)*d2:i*d2,1).';
end
end
```

### Code B.7 The implementation of a NASS state for 2D grayscale images in (3.58)

```
% f is a grayscale image.
% The function GQ2 transforms the grayscale image f into a NASS
state, i.e., a vector Q.
function [G,Q]=GQ2(f)
[ row,col]=size(f);
M=256;
fg=pi()*f/(2*M-2); % Convert colors to angles
dg=abs(fg).*abs(fg);
G=sqrt(sum(dg(:)));
%Normalize the image
g = fg/G;
%Convert a 2D image to a column vector
Q=d2tod1(g);
end
```

### Code B.8 Retrieving a 2D grayscale image from a NASS state

```
% Q is a vector for a NASS state.
% G is a normalization factor.
% The function QG2 converts a vector Q to a row*col grayscale
image f.
function f=QG2(Q, G, row, col)
g = Q/G;
f1=d1tod2(g, row, col);
% Convert angles to colors
f=f1*(2*256-2)/pi();
end
```

**Code B.9** The implementation of a NASS state for 2D color images in (3.58)

```

% f is a color image.
% The function CQ2 transforms the color image f into a NASS
state, i.e., a vector Q.
function [G,Q]=CQ2(f)
[row,col,tem]=size(f);
%Convert colors to angles
M=2^24;
fg=pi()*(f(:,:,1)*256*256 + f(:,:,2)*256 + f(:,:,3))/(2*M-2);
dg=abs(fg).*abs(fg);
G=sqrt(sum(dg(:)));
%Normalize the image
g = fg/G;
%Convert a 2D image to a column vector
Q=d2tod1(g);
end

```

**Code B.10** Retrieving a 2D color image from a NASS state

```

% Q is a vector for a NASS state.
% G is a normalization factor.
% The function QC2 converts a vector Q to a row*col color image
f.
function gf =QC2(Q, G, row, col)
g = Q/G;
f=d1tod2(g, row, col);
% Convert angles to colors
M=2^24;
wginvac=f*(2*M-2)/pi();
gf(:,:,1)=fix(wginvac/(256*256));
gf(:,:,2)=rem(fix(wginvac/256),256);
gf(:,:,3)=rem(wginvac,256);
end

```

**Code B.11** The implementation of a NASS state for a video in (3.60)

```

% f is a vedio.
% The function GQ3 transforms the video f into a NASS state,
i.e., a vector Q.
function [G,Q]=GQ3(f)
[row,col,st]=size(f);
% Convert colors to angles
M=2^8;
fg= pi()*f/(2*M-2);
dg=abs(fg).*abs(fg);
G=sqrt(sum(dg(:)));

```

## Appendix 2

```
% Normalize the image
g = fg/G;
% Convert a 3D image to a column vector
Q=[];
for i=1:row
    A=zeros(col,st);
    for j=1:col
        for k=1:st
            A(j,k)=g(i,j,k);
        end
    end
    Q=[Q;d2tod1(A)];
end
end
```

### Code B.12 Retrieving a video from a NASS sate

```
% Q is a vector for a NASS state.
% G is a normalization factor.
% The function QG3 converts a vector Q to a row*col*st video f.
function f =QG3(Q, G, row, col, st)
ghx=zeros(row,col,st);
for i=1:row
    i1=(i-1)*col*st+1;
    i2=i*col*st;
    A1=Q(i1:i2,1);
    A=d1tod2(A1, col, st);
    for j=1:col
        for k=1:st
            ghx(i,j,k)=A(j,k);
        end
    end
end
end
wginva=ghx*G;
% Convert angles to colors
M=2^8;
f=wginva*(2*M-2)/pi();
end
```

## The Codes for Geometric Transformations in Chapter 4

**Code B.13** The code of the two-point swapping for the grayscale image in (4.29)

```
% Two-point swapping for the grayscale image in (4.29)
function []=Two-pointSwapImage()
clear all
clc
% f is a 8*4 grayscale image
f=[1 2 3 4];
for i=1:7
    f=[f;i*4+1 i*4+2 i*4+3 i*4+4;];
end
[row, col]=size(f);
% Call the function GQ2 to create a NASS state
[G,Q]=GQ2(f);
% Calculate the matrix TP of the circuit in Figure 4.1
W=[0 1; -1 0];
W1=[0 -1; 1 0];
X=[0 1; 1 0];
TP=Cun(4,15,X)*Cup(1,0,Cun(3,7,W))*Cup(2,0,Cun(2,3,W))*Cup(3,0,
Cun(1,1,W))*Cup(4,0,W);
TP=Cup(4,0,W1)*Cup(3,0,Cun(1,1,W1))*Cup(2,0,Cun(2,3,W1))*Cup(1,
0,Cun(3,7,W1))*TP;
%Implement the two-point swapping
TPQ=TP*Q;
% Retrieve the transformed image
Tf=uint8(QG2(TPQ, G, row, col));
end
```

**Code B.14** The code of the two-point swapping for the video in Figure 4.21

```
% Two-point swapping for the video in Figure 4.21
function []=Two-pointSwapVideo()
clear all
clc
% f is a 4*2*4 video
f1=[1 5; 9 13; 17 21; 25 29];
f(:,:,1)=f1;
f(:,:,2)=f1+1;
f(:,:,3)=f1+2;
f(:,:,4)=f1+3;
[row,col,st]=size(f);
% Call the function GQ3 to create a NASS state
[G,Q]=GQ3(f);
% Calculate the matrix TP of the circuit in Figure 4.1
```

## Appendix 2

```
W=[0 1; -1 0];
W1=[0 -1; 1 0];
X=[0 1; 1 0];
TP=Cun(4,15,X)*Cup(1,0,Cun(3,7,W))*Cup(2,0,Cun(2,3,W))*Cup(3,0,
Cun(1,1,W))*Cup(4,0,W);
TP=Cup(4,0,W1)*Cup(3,0,Cun(1,1,W1))*Cup(2,0,Cun(2,3,W1))*Cup(1,
0,Cun(3,7,W1))*TP;
TPQ=TP*Q; % Implement the two-point swapping
Tf=uint8(QG3(TPQ, G, row, col, st)); % Retrieve the transformed
image
end
```

**Code B.15** The code of symmetric flip transformations for the image in Figure 4.22

```
% Symmetric flip transformations for the 128*128 color image
function []=FlipImage()
clear all
clc
X=[0 1; 1 0];
I=[1 0; 0 1];
% f is a 128*128 color image
f = double(imread('airplane.bmp'));
subplot(1,3,1),imshow(uint8(f));
[row, col,tem]=size(f);
r=log2(row);
c=log2(col);
% Call the function CQ2 to create a NASS state
[G,Q]=CQ2(f);
% Flip along x axis
GFX=kron(tenProduct(I,r),tenProduct(X,c));
GFXQ=GFX*Q;
% Flip along y axis
GFY=kron(tenProduct(X,r),tenProduct(I,c));
GFYQ=GFY*Q;
% Retrieve the transformed image
fx=QC2(GFXQ, G, row, col);
fy=QC2(GFYQ, G, row, col);
subplot(1,3,2),imshow(uint8(fx));
subplot(1,3,3),imshow(uint8(fy));
end
```

**Code B.16** The code of symmetric flip transformations for the video in Figure 4.23

```
% Symmetric flip for the 128*128*2 video
function []=FlipVideo()
```

```

clear all
clc
X=[0 1; 1 0];
I=[1 0; 0 1];
% f is a 128*128*2 video
for i=1:2
    A=imread([num2str(i),'.bmp']);
    f(:,:,i)=double(A);
    subplot(2,4,i);
    imshow(A,[]);
end
[row, col,st]=size(f);
r=log2(row);
c=log2(col);
t=log2(st);
% Call the function GQ3 to create a NASS state
[G,Q]=GQ3(f);
% Flip along x axis
GFX=kron(tenProduct(I,r),tenProduct(X,c+t));
GFXQ=GFX*Q;
% Flip along y axis
GFY=kron(kron(tenProduct(X,r),tenProduct(I,c)),tenProduct(X,t));
GFYQ=GFY*Q;
% Flip along t axis
GFT=kron(tenProduct(X,r+c),tenProduct(I,t));
GFTQ=GFT*Q;
% Retrieve the transformed video
fx=QG3(GFXQ, G, row, col, st);
for i=1:2
    subplot(2,4,2+i);
    A=fx(:,:,i);
    imshow(uint8(A),[]);
end
fy=QG3(GFYQ, G, row, col, st);
for i=1:2
    subplot(2,4,4+i);
    A=fy(:,:,i);
    imshow(uint8(A),[]);
end
ft=QG3(GFTQ, G, row, col, st);
for i=1:2
    subplot(2,4,6+i);
    A=ft(:,:,i);
    imshow(uint8(A),[]);
end
end
end

```

## Appendix 2

### Code B.17 The code of local flip transformations for the image in Figure 4.24

```
% Local flip for the 128*128 color image
function []=LocalFlipImage()
clear all
clc
X=[0 1; 1 0];
I=[1 0; 0 1];
% f is a 128*128 color image
f = double(imread('airplane.bmp'));
subplot(1,3,1),imshow(uint8(f));
[row, col,tem]=size(f);
r=log2(row);
c=log2(col);
% Call the function CQ2 to create a NASS state
[G,Q]=CQ2(f);
% Local flip along x axis for the left part of the image
m=0;
GFXL=kron(tenProduct(I,r),Cup(1,m,tenProduct(X,c-1)));
GFXLQ=GFXL*Q;
% Local flip along x axis for the right part of the image
m=1;
GFXR=kron(tenProduct(I,r),Cup(1,m,tenProduct(X,c-1)));
GFXRQ=GFXR*Q;
% Retrieve the transformed image
fl=QC2(GFXLQ, G, row, col);
fr=QC2(GFXRQ, G, row, col);
subplot(1,3,2),imshow(uint8(fl));
subplot(1,3,3),imshow(uint8(fr));
end
```

### Code B.18 The code of local flip transformations for the video in Figure 4.25

```
% Local flip for the 128*128*2 video
function []=LocalFlipVideo()
clear all
clc
X=[0 1; 1 0];
I=[1 0; 0 1];
% f is a 128*128*2 video
for i=1:2
    A=imread([num2str(i),'.bmp']);
    f(:, :, i)=double(A);
    subplot(1,6,i);
    imshow(A, []);
end
[row, col,st]=size(f);
% Call the function GQ3 to create a NASS state
```

```

[G,Q]=GQ3(f);
% Local flip along t axis for the left part of the image frames
temp=kron(Cun(1,0,tenProduct(X,7)),tenProduct(I,7))*Q;
GFTLQ=kron(tenProduct(I,7),kron(Cup(1,0,tenProduct(X,6)),I))*t
emp;
% Local flip along t axis for the right part of the image
frames
temp=kron(Cun(1,1,tenProduct(X,7)),tenProduct(I,7))*Q;
GFTUQ=kron(tenProduct(I,7),kron(Cup(1,1,tenProduct(X,6)),I))*t
emp;
% Retrieve the transformed video
fl=QG3(GFTLQ, G, row, col, st);
for i=1:2
    subplot(1,6,2+i);
    A=fl(:, :, i);
    imshow(uint8(A), []);
end
fu=QG3(GFTUQ, G, row, col, st);
for i=1:2
    subplot(1,6,4+i);
    A=fu(:, :, i);
    imshow(uint8(A), []);
end
end
end

```

**Code B.19** The implementation of an  $n$ -qubit Swap gate in Figure 4.26.

```

% Swap gate with  $n$  qubits ( $n \geq 2$ )
function P=Swa(n)
I=[1 0;0 1];
X=[0 1; 1 0];
if n<2
    P=0;
else
    Im=1;
    if n>2
        Im=tenProduct(I,n-2);
    end
    P=Cun(1,1,kron(X,Im))*Cup(1,1,kron(Im,X))*Cun(1,1,kron(X,
Im));
end
end
end

```

## Appendix 2

### Code B.20 The code of orthogonal rotation transformations for the image in Figure 4.27

```
% Orthogonal rotation for the 128*128 color image
function []=OrthogRotationImage()
clear all
clc
X=[0 1; 1 0];
I=[1 0; 0 1];
% f is a 128*128 color image
f = double(imread('airplane.bmp'));
subplot(1,4,1),imshow(uint8(f));
[row, col,tem]=size(f);
r=log2(row);
c=log2(col);
% Call the function CQ2 to create a NASS state
[G,Q]=CQ2(f);
% Orthogonal rotation with pi/2
temp=kron(tenProduct(X,r),tenProduct(I,c))*Q;
temp=kron(Swa(r+1),tenProduct(I,c-1))*temp;
for i=r-2:-1:1
    temp=kron(tenProduct(I,r-i-1),kron(Swa(r+1),tenProduct(I,i)
    ))*temp;
end
GO1=kron(tenProduct(I,r-1),Swa(r+1))*temp;
% Orthogonal rotation with pi
GO2=tenProduct(X,r+c)*Q;
% Orthogonal rotation with 3*pi/2
temp=kron(tenProduct(I,c),tenProduct(X,r))*Q;
temp=kron(Swa(r+1),tenProduct(I,c-1))*temp;
for i=r-2:-1:1
    temp=kron(tenProduct(I,r-i-1),kron(Swa(r+1),tenProduct(I,i)
    ))*temp;
end
GO3=kron(tenProduct(I,r-1),Swa(r+1))*temp;
% Retrieve the transformed image
f1=QC2(GO1, G, row, col);
f2=QC2(GO2, G, row, col);
f3=QC2(GO3, G, row, col);
subplot(1,4,2),imshow(uint8(f1));
subplot(1,4,3),imshow(uint8(f2));
subplot(1,4,4),imshow(uint8(f3));
end
```

**Code B.21** The code of orthogonal rotation transformations for the video in Figure 4.28

```
% Orthogonal rotation for the 128*128*2 video
function []=OrthogRotationVideo()
clear all
clc
X=[0 1; 1 0];
I=[1 0; 0 1];
% f is a 128*128*2 video
for i=1:2
    A=imread([num2str(i),'.bmp']);
    f(:, :, i)=double(A);
    subplot(2,4,i);
    imshow(A, []);
end
[row, col, st]=size(f);
r=log2(row);
c=log2(col);
t=log2(st);
% Call the function GQ3 to create a NASS state
[G,Q]=GQ3(f);
% Orthogonal rotation with pi/2
GO1=kron(tenProduct(X,r),tenProduct(I,c+1))*Q;
GO1=kron(Swa(r+1),tenProduct(I,c))*GO1;
for i=r-2:-1:0
    GO1=kron(tenProduct(I,r-i-1),kron(Swa(r+1),tenProduct(I,i+1
    )))*GO1;
end
% Orthogonal rotation with pi
GO2=kron(tenProduct(X,r+c),I)*Q;
% Orthogonal rotation with 3*pi/2
GO3=kron(kron(tenProduct(I,r),tenProduct(X,c)),I)*Q;
GO3=kron(Swa(r+1),tenProduct(I,c))*GO3;
for i=r-2:-1:0
    GO3=kron(tenProduct(I,r-i-1),kron(Swa(r+1),tenProduct(I,i+1
    )))*GO3;
end
% Retrieve the transformed image
f1=QG3(GO1, G, row, col, st);
for i=1:2
    subplot(2,4,2+i);
    A=f1(:, :, i);
    imshow(uint8(A), []);
end
f2=QG3(GO2, G, row, col, st);
for i=1:2
    subplot(2,4,4+i);
```

## Appendix 2

```
A=f2(:,:,i);
imshow(uint8(A),[]);
end
f3=QG3(GO3, G, row, col, st);
for i=1:2
    subplot(2,4,6+i);
    A=f3(:,:,i);
    imshow(uint8(A),[]);
end
end
```

**Code B.22** The code for the circuit of  $T_k$  in Figure 4.18 (a)

```
function P=Tk(k) % k>1
X=[0 1; 1 0];
I=[1 0; 0 1];
if k==1
    P=X;
else
    P=Cun(k-1,2^(k-1)-1,X);
    for i=1:k-2
        P=kron(tenProduct(I,i),Cun(k-1-i,2^(k-1-i)-1,X))*P;
    end
    P=kron(tenProduct(I,k-1),X)*P;
end
end
```

**Code B.23** The code of translation transformations for the image in Figure 4.29

```
% Translation for the 128*128 color image
function []=TranslationImage()
clear all
clc
X=[0 1; 1 0];
I=[1 0; 0 1];
% f is a 128*128 color image
f = double(imread('airplane.bmp'));
subplot(1,3,1),imshow(uint8(f));
[row, col,tem]=size(f);
r=log2(row);
c=log2(col);
% Call the fuction CQ2 to create a NASS state
[G,Q]=CQ2(f);
% Call the function Tk to realize translation along x axis
TX=kron(Tk(r),tenProduct(I,c));
% TX is applied repeatedly 10 times to translate 10 pixels
along x axis
```

```

TXQ=TX*Q;
for i=1:9
    TXQ=TX*TXQ;
end
% Call the function Tk to realize translation along y axis
TY=kron(tenProduct(I,r),Tk(c));
% TY is applied repeatedly 10 times to translate 10 pixels
along y axis
TYQ=TY*Q;
for i=1:9
    TYQ=TY*TYQ;
end
% Retrieve the transformed image
fx=QC2(TXQ, G, row, col);
fy=QC2(TYQ, G, row, col);
subplot(1,3,2),imshow(uint8(fx));
subplot(1,3,3),imshow(uint8(fy));
end

```

**Code B.24** The code of the translation transformation for the video in Figure 4.30

```

% Translation for the 128*128*2 video
function []=TranslationVideo()
clear all
clc
X=[0 1; 1 0];
I=[1 0; 0 1];
% f is a 128*128*2 video
for i=1:2
    A=imread([num2str(i),'.bmp']);
    f(:,:,i)=double(A);
    subplot(1,4,i);
    imshow(A,[]);
end
[row, col,st]=size(f);
r=log2(row);
c=log2(col);
t=log2(st);
% Call the function GQ3 to create a NASS state
[G,Q]=GQ3(f);
% Call the function Tk to realize translation
THQ=kron(tenProduct(I,r+c),Tk(t))*Q;
% Retrieve the transformed video
fq=QG3(THQ, G, row, col, st);
for i=1:2
    subplot(2,4,2+i);
    A=fq(:,:,i);

```

## Appendix 2

```
        imshow(uint8(A), []);  
end  
end
```

## The Codes for Quantum Fourier Transforms in Chapter 5

**Code B.25** The code for the circuit of  $\Gamma_{2^n}$  in Figure 5.2 for  $n > 1$

```
function SP=allpermutation(n)  
    r=fix(n/2);  
    I=[1 0;0 1];  
    SP=Swa(n);  
    if n>3  
        for i=2:r  
            In=tenProduct(I,i-1);  
            SP=SP*kron(kron(In,Swa(n-(i-1)*2)),In);  
        end  
    end  
end
```

**Code B.26** The code for the circuit of  $P_{2^{n-1},2}$  in Figure 5.3 for  $n > 1$

```
function SP=Permutationn2(n)  
    I=[1 0;0 1];  
    if n==2  
        SP=Swa(2);  
    else  
        SP=allpermutation(n)*kron(allpermutation(n-1),I);  
    end  
end
```

**Code B.27** The code for the circuit of  $P_{2,2^{n-1}}$  in Figure 5.3 for  $n > 1$

```
function SP=Permutation2n(n)  
    I=[1 0;0 1];  
    if n==2  
        SP=Swa(2);  
    else  
        SP=kron(allpermutation(n-1),I)*allpermutation(n);  
    end  
end
```

**Code B.28** The code for the circuit of  $S_{2^n}(R_n)$  in Figure 5.11 for  $n > 1$

```
function P=S2N(n,Rn)
I=[1 0;0 1];
P=Cun(1,1,kron(Rn,tenProduct(I,n-2)));
for j=1:n-2
    Rn=Rn*Rn;
    P=kron(Cun(1,1,kron(Rn,tenProduct(I,n-2-
j))),tenProduct(I,j))*P;
end
end
```

**Code B.29** The code for the circuit of  $T_{2^n}(R_n)$  in Figure 5.12 for  $n > 1$

```
function P=T2N(n,Rn)
I=[1 0;0 1];
P=kron(tenProduct(I,n-2),Cup(1,1,Rn));
for j=1:n-2
    Rn=Rn*Rn;
    P=P*kron(tenProduct(I,n-2-j),Cup(1,1,kron(tenProduct(I,j),
Rn)));
end
end
```

**Code B.30** The code for the first circuit of 1D-QFT and its inverse in Figure 5.13 and Figure 5.14

```
% Rn=[1 0; 0 exp(2*pi()*i/2^n)] and Rn=[1 0; 0 exp(-
2*pi()*i/2^n)] correspond to 1D-QFT and its inverse,
respectively.
function F=ODQFT1(n,Rn)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0;0 1];
if n==1
    F=H;
else
    F=kron(H,tenProduct(I,n-1))*S2N(n,Rn)*kron(I,ODQFT1(n-
1,Rn*Rn))*Permutationn2(n);
end
end
```

## Appendix 2

**Code B.31** The code for the second circuit of 1D-QFT and its inverse in Figure 5.15 and Figure 5.16

```
% Rn=[1 0; 0 exp(2*pi()*i/2^n)] and Rn=[1 0; 0 exp(-
2*pi()*i/2^n)] correspond to 1D-QFT and its inverse,
respectively.
function F=ODQFT2(n,Rn)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0;0 1];
if n==1
    F=H;
else
    F=Permutation2n(n)*kron(I,ODQFT2(n-1,Rn*Rn))*S2N(n,Rn)*
kron(H,tenProduct(I,n-1));
end
end
```

**Code B.32** The code for the test of 1D-QFT with n=4

```
% Rn=[1 0; 0 exp(2*pi()*i/2^n)] and Rn=[1 0; 0 exp(-
2*pi()*i/2^n)] correspond to 1D-QFT and its inverse,
respectively.
function F=ODQFT3(n,Rn)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0;0 1];
if n==1
    F=H;
else
    F=kron(ODQFT3(n-1,Rn*Rn),I)*T2N(n,Rn)*kron(tenProduct(I,n-
1),H)*Permutation2n(n);
end
end
```

**Code B.33** The code for the fourth circuit of the inverse of 1D-QFT in Figure 5.19 and Figure 5.20

```
% Rn=[1 0; 0 exp(2*pi()*i/2^n)] and Rn=[1 0; 0 exp(-
2*pi()*i/2^n)] correspond to 1D-QFT and its inverse,
respectively.
function F=ODQFT4(n,Rn)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0;0 1];
if n==1
    F=H;
else
    F=Permutationn2(n)*kron(tenProduct(I,n-1),H)*T2N(n,Rn)*
```

```
kron(ODQFT4(n-1,Rn*Rn),I);
end
end
```

### Code B.34 The code for the test of 1D-QFTs with $n=4$

```
function []=testODFT()
clear all
clc
% f is a 16*1 column vector
f=[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16]';
n=4;
Rn=[1 0; 0 exp(2*pi()*i/2^n)];
IRn=[1 0; 0 exp(-2*pi()*i/2^n)];
% Apply inverse classical Fourier transform on the column
vector
FT=sqrt(2^n)*ifft(f);
% Apply 1D-QFT for the first circuit in Figure 5.13 on the
column vector
QFT1=ODQFT1(n,Rn)*f;
disp('The first circuit');
norm(abs(FT-QFT1))
% Apply the inverse of 1D-QFT for the first circuit in Figure
5.14 on QFT1
IQFT1=ODQFT1(n,IRn)*QFT1;
% Compare IQFT1 with f
norm(abs(IQFT1-f))
% Apply 1D-QFT for the second circuit in Figure 5.15 on the
column vector
QFT2=ODQFT2(n,Rn)*f;
disp('The second circuit');
norm(abs(FT-QFT2))
% Apply the inverse of 1D-QFT for the second circuit in Figure
5.16 on QFT2
IQFT2=ODQFT2(n,IRn)*QFT2;
% Compare IQFT2 with f
norm(abs(IQFT2-f))
% Apply 1D-QFT for the third circuit in Figure 5.17 on the
column vector
QFT3=ODQFT3(n,Rn)*f;
disp('The third circuit');
norm(abs(FT-QFT3))
% Apply the inverse of 1D-QFT for the third circuit in Figure
5.18 on QFT3
IQFT3=ODQFT3(n,IRn)*QFT3;
% Compare IQFT3 with f
norm(abs(IQFT3-f))
% Apply 1D-QFT for the fourth circuit in Figure 5.19 on the
```

## Appendix 2

```
column vector
QFT4=ODQFT4(n,Rn)*f;
disp('The third circuit');
norm(abs(FT-QFT4))
% Apply the inverse of 1D-QFT for the fourth circuit in Figure
5.20 on QFT4
IQFT4=ODQFT4(n,IRn)*QFT4;
% Compare IQFT4 with f
norm(abs(IQFT4-f))
end
```

### Code B.35 The code for the circuits of 2D-QFT and its inverse in Figure 5.25

```
function F=TDQFT(m,Rm,k,Rk)
F=kron(ODQFT1(m,Rm),ODQFT1(k,Rk));
end
```

### Code B.36 The code for the test of 2D-QFT with a $128 \times 128$ image

```
function []=testTODFT()
clear all
clc
% f is a 128*128 image
f = double(imread('gclena128.bmp'));
subplot(1,2,1);
imshow(uint8(f),[]); %The original image
% Apply classical 2D inverse Fourier transform on the image
[row,col]=size(f);
m=log2(row);
k=log2(col);
FT=sqrt(2^m)*sqrt(2^k)*ifft2(f);
% Call the function GQ2 to create a NASS state
[G,Q]=GQ2(f);
% Apply 2D-QFT in Figure 5.25 on the image
Rm=[1 0; 0 exp(2*pi()*i/2^m)];
Rk=[1 0; 0 exp(2*pi()*i/2^k)];
QF=TDQFT(m,Rm,k,Rk)*Q;
% Retrieve the transformed image
QFI=QG2(QF, G, row, col);
% Display the amplitude spectrum of the transformed image
subplot(1,2,2);
mesh(abs(QFI))
% Compare classical 2D inverse Fourier transform with 2D-QFT
norm(abs(FT-QFI))
% Apply the inverse of 2D-QFT on the transformed image QF
IRm=[1 0; 0 exp(-2*pi()*i/2^m)];
IRk=[1 0; 0 exp(-2*pi()*i/2^k)];
```

```

IQFI=TDQFT(m,IRm,k,IRk)*QF;
% Retrieve the image
IQFTI=QG2(IQFI, G, row, col);
% Compare IQFTI with f
norm(abs(IQFTI-f))
end

```

**Code B.37** The code for the circuits of 3D-QFT and its inverse in Figure 5.26

```

function F=ThDQFT(m,Rm,k,Rk,h,Rh)
F=kron(kron(ODQFT1(m,Rm),ODQFT1(k,Rk)),ODQFT1(h,Rh));
end

```

**Code B.38** The code for the test of 3D-QFT with a 64\*32\*8 video

```

function []=testThDQFT()
clear all
clc
% f is a 128*128*2 video
for j=1:2
    A=imread([num2str(j),'.bmp']);
    f(:, :, j)=double(A);
    subplot(1,4,j);
    imshow(A, []);
end
% Apply classical 3D inverse Fourier transform on the video
[row, col, st]=size(f);
FT=sqrt(row)*sqrt(col)*sqrt(st)*ifftn(f);
% Call the function GQ3 to create a NASS state
m=log2(row);
k=log2(col);
h=log2(st);
[G,Q]=GQ3(f);
% Apply 3D-QFT in Figure 5.26 on the image
Rm=[1 0; 0 exp(2*pi()*i/2^m)];
Rk=[1 0; 0 exp(2*pi()*i/2^k)];
Rh=[1 0; 0 exp(2*pi()*i/2^h)];
QF=ThDQFT(m,Rm,k,Rk,h,Rh)*Q;
% Retrieve the transformed video
QFV=QG3(QF, G, row, col, st);
% Display the amplitude spectrum of the image frames of the
transformed video
for j=1:2
    subplot(1,4,2+j);
    A=QFV(:, :, j);
    mesh(abs(A))
end

```

## Appendix 2

```
% Compare classcial 3D inverse Fourier transform with 3D-QFT
for j=1:st
    norm(abs(FT(:, :, j)-QFV(:, :, j)))
end
% Apply the inverse of 3D-QFT on the transformed video QF
IRm=[1 0; 0 exp(-2*pi()*i/2^m)];
IRk=[1 0; 0 exp(-2*pi()*i/2^k)];
IRh=[1 0; 0 exp(-2*pi()*i/2^h)];
IQF=ThDQFT(m, IRm, k, IRk, h, IRh)*QF;
% Retrieve the video
IQFV=QG3(IQF, G, row, col, st);
% Compare IQFTI with f
for j=1:st
    norm(abs(f(:, :, j)-IQFV(:, :, j)))
end
end
```

## The Codes for Quantum Wavelet Transforms in Chapter 6

### Code B.39 The code for the Doppler function

```
function s=d(t)
    s=sqrt(t*(1-t))*sin(2*pi()*1.05/(t+0.05));
end
```

### Code B.40 The code for the implementation of HQWT in Figure 6.3 and Figure 6.5

```
% k>=0 and n>k
function F=FHNK(n,k)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0; 0 1];
if k==0
    if n==1
        F=H;% The initial value in Figure 6.5
    else
        F=Permutationn2(n)*kron(tenProduct(I,n-1),H); % The
initial value in Figure 6.3
    end
else
    % The (k+1)-level HQWT
    F=Cup(1,0,FHNK(n-1,k-1))*Permutationn2(n)*kron(tenProduct(I
,n-1),H);
end
end
```

**Code B.41** The code for the implementation of HQWT in Figure 6.4 and Figure 6.6

```
% k>=0 and n>k
function F=SHNK(n,k)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0;0 1];
if k==0
    if n==1
        F=H;% The initial value in Figure 6.6
    else
        F=Permutationn2(n)*kron(tenProduct(I,n-1),H); % The
initial value in Figure 6.4
    end
else
    % The (k+1)-level HQWT
    F=Permutationn2(n)*Cun(1,0,SHNK(n-1,k-
1))*kron(tenProduct(I,n-1),H);
end
end
```

**Code B.42** The code for the implementation of the inverse of HQWT in (6.16)

```
% k>=0 and n>k
function F=IFHNK(n,k)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0;0 1];
if k==0
    if n==1
        F=H;% The initial value of IFHNK(n,n-1)
    else
        F=kron(tenProduct(I,n-1),H)*Permutation2n(n); % The
initial value of IFHNK(n,k) with k<n-1
    end
else
    % The inverse of the (k+1)-level HQWT
    F=kron(tenProduct(I,n-1),H)*Permutation2n(n)*Cup(1,0,IFHNK
(n-1,k-1));
end
end
```

**Code B.43** The code for the implementation of the inverse of HQWT in (6.17)

```
% k>=0 and n>k
function F=ISHNK(n,k)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0;0 1];
```

## Appendix 2

```
if k==0
    if n==1
        F=H;% The initial value of ISHNK(n,n-1)
    else
        F=kron(tenProduct(I,n-1),H)*Permutation2n(n); % The
        initial value of ISHNK(n,k) with k<n-1
    end
else
    % The inverse of the (k+1)-level HQWT
    F=kron(tenProduct(I,n-1),H)*Cun(1,0,ISHNK(n-1,k-
    1))*Permutation2n(n);
end
end
```

**Code B.44** The code for the first 3 levels of HQWT with the input V in (6.46)

```
function []=testFirst3LevelHaar()
clc
clear all
for k=1:2048
    V(1,k)=d((k-1)/2048);
end
x=1:2048;
m=11;
V1=V';
% 0 level
subplot(4,8,1:8);
plot(x,V1',' '.');
% 1 level
subplot(4,8,9:12);
V1=FHNK(m,0)*V';
x=1:1024;
plot(x,V1(x,1)', ' '.');
subplot(4,8,13:16)
x=1025:2048;
plot(x,V1(x,1)', ' '.');
% 2 levels
V1=FHNK(m,1)*V';
subplot(4,8,17:18);
x=1:512;
plot(x,V1(x,1)', ' '.');
subplot(4,8,19:20);
x=513:1024;
plot(x,V1(x,1)', ' '.');
subplot(4,8,21:24);
x=1025:2048;
plot(x,V1(x,1)', ' '.');
% 3 levels
```

```

V1=FHNK(m,2)*V';
subplot(4,8,25);
x=1:256;
plot(x,V1(x,1)',','.');
subplot(4,8,26);
x=257:512;
plot(x,V1(x,1)',','.');
subplot(4,8,27:28);
x=513:1024;
plot(x,V1(x,1)',','.');
subplot(4,8,29:32);
x=1025:2048;
plot(x,V1(x,1)',','.');
end

```

**Code B.45** The code for all levels of HQWT with the input  $V$  in (6.46)

```

function []=testAllLevelHaar()
clc
clear all
for k=1:2048
    V(1,k)=d((k-1)/2048);
end
x=1:2048;
m=11;
V1=V';
% 0-11 levels
subplot(12,1,1)
plot(x,V1',','.');
for j=1:m
    V1=FHNK(m,j-1)*V'; % Apply HQWT in Figure 6.3 and Figure
6.5 on the vector V
    subplot(12,1,j+1)
    plot(x,V1',','.');
end
end

```

**Code B.46** The code for the comparisons of HQWTs, their inverses, and classical Haar wavelet transform

```

function []=testComparisonHaar()
clc
clear all
for k=1:2048
    V(1,k)=d((k-1)/2048);
end
x=1:2048;

```

## Appendix 2

```

m=11;
% 0-11 levels
for j=1:m
    % Apply HQWT in Figure 6.3 and Figure 6.5 on the vector V'
    V1=FHNK(m,j-1)*V';
    % Apply HQWT in Figure 6.4 and Figure 6.6 on the vector V'
    V2=SHNK(m,j-1)*V';
    % Apply classical Haar wavelet transform on the vector V'
    [C2,L2] = wavedec(V',j,'haar');
    %The comparisons of HQWTs and classical Haar wavelet
transform
    err1(j,1) = norm(V1-C2);
    err2(j,1) = norm(V2-C2);
    % Apply the inverse of HQWT in (6.16) on the vector V1
    V3=IFHNK(m,j-1)*V1;
    %The comparisons of V3 and V'
    err3(j,1) = norm(V3-V');
    % Apply the inverse of HQWT in (6.17) on the vector V2
    V4=ISHNK(m,j-1)*V2;
    %The comparisons of V4 and V'
    err4(j,1) = norm(V4-V');
end
end

```

**Code B.47** The code for the circuit of the downshift matrix  $Q_{2^n}$  in Figure 6.9 (a)

```

function P=Q2N(n)
I=[1 0;0 1];
X=[0 1; 1 0];
if n==1
    P=X;
else
    P=kron(tenProduct(I,n-1),X)*Cun(1,0,Q2N(n-1));
end

```

**Code B.48** The code for the circuit of  $(Q_{2^n})^{-1}$  in Figure 6.10 (a)

```

function P=IQ2N(n)
I=[1 0;0 1];
X=[0 1; 1 0];
if n==1
    P=X;
else
    P=Cun(1,0,IQ2N(n-1))*kron(tenProduct(I,n-1),X);
end

```

**Code B.49** The code for the circuit of the matrix  $D_{2^n}$  in Figure 6.9 (b)

```
% n>1
function P=D2N(n)
I=[1 0;0 1];
a1=5*pi()/12;
a0=2*pi()/3;
S0=[sin(a0) cos(a0);cos(a0) -1*sin(a0)];
S1=[-sin(a1) cos(a1);cos(a1) sin(a1)];
P=kron(tenProduct(I,n-1),S1)*Q2N(n)*kron(tenProduct(I,n-1),S0);
end
```

**Code B.50** The code for the circuit of the matrix  $(D_{2^n})^{-1}$  in Figure 6.10 (b)

```
% n>1
function P=ID2N(n)
I=[1 0;0 1];
a1=5*pi()/12;
a0=2*pi()/3;
S0=[sin(a0) cos(a0);cos(a0) -1*sin(a0)];
S1=[-sin(a1) cos(a1);cos(a1) sin(a1)];
P=kron(tenProduct(I,n-1),S0)*IQ2N(n)*kron(tenProduct(I,n-1),S1);
end
```

**Code B.51** The code for the circuit of  $D_{2^n}^P$  in Figure 6.11 (a)

```
% n>1
function P=D2NP(n)
P=D2N(n)*IQ2N(n);
end
```

**Code B.52** The code for the circuit of  $(D_{2^n}^P)^{-1}$  in Figure 6.11 (b)

```
% n>1
function P=ID2NP(n)
P=Q2N(n)*ID2N(n);
end
```

**Code B.53** The code for the circuit of D4QWT  $F_{2^n}^k$  in Figure 6.12

```
% k>=0 and k<=n-2
function F=F2NK(n,k)
if k==0
    F=Permutationn2(n)*D2NP(n); % The initial value
```

## Appendix 2

```
else
    % The (k+1)-level D4QWT
    F=Permutationn2(n)*Cun(1,0,F2NK(n-1,k-1))*D2NP(n);
end
end
```

**Code B.54** The code for the circuit of the inverse of D4QWT  $(F_{2^n}^k)^{-1}$  in Figure 6.13

```
% k>=0 and k<=n-2
function F=IF2NK(n,k)
if k==0
    F=ID2NP(n)*Permutation2n(n); % The initial value
else
    % The invese of the (k+1)-level D4QWT
    F=ID2NP(n)*Cun(1,0,IF2NK(n-1,k-1))*Permutation2n(n);
end
end
```

**Code B.55** The code for the first 3 levels of D4QWT with the input V in (6.46)

```
function []=testFirst3LevelD4()
clc
clear all
% Call Doppler function d()
for k=1:2048
    V(1,k)=d((k-1)/2048);
end
x=1:2048;
m=11;
V1=V';
set(gcf,'color','white');
% 0 level
subplot(4,8,1:8);
x=1:2048;
plot(x,V','');
% 1 level
subplot(4,8,9:12);
V1=F2NK(m,0)*V';
x=1:1024;
plot(x,V1(x,1)','');
subplot(4,8,13:16)
x=1025:2048;
plot(x,V1(x,1)','');
% 2 levels
V1=F2NK(m,1)*V';
subplot(4,8,17:18);
```

```

x=1:512;
plot(x,V1(x,1)',','.');
subplot(4,8,19:20);
x=513:1024;
plot(x,V1(x,1)',','.');
subplot(4,8,21:24);
x=1025:2048;
plot(x,V1(x,1)',','.');
% 3 levels
V1=F2NK(m,2)*V';
subplot(4,8,25);
x=1:256;
plot(x,V1(x,1)',','.');
subplot(4,8,26);
x=257:512;
plot(x,V1(x,1)',','.');
subplot(4,8,27:28);
x=513:1024;
plot(x,V1(x,1)',','.');
subplot(4,8,29:32);
x=1025:2048;
plot(x,V1(x,1)',','.');
end

```

**Code B.56** The code for all levels of D4QWT with the input  $V$  in (6.46)

```

function []=testAllLevelD4()
clc
clear all
% Call Doppler function d()
for k=1:2048
    V(1,k)=d((k-1)/2048);
end
x=1:2048;
m=11;
V1=V';
% 0-10 levels
subplot(11,1,1)
plot(x,V1',','.');
for j=1:m-1
    % Apply D4QWT in Figure 6.12 on the vector V
    V1=F2NK(m,j-1)*V';
    subplot(11,1,j+1)
    plot(x,V1',','.');
end
end

```

## Appendix 2

### Code B.57 The code for the comparisons of D4QWT, its inverse, and classical D4 wavelet transform

```
function []=testComparisonD4()
clc
clear all
% Call Doppler function d()
for k=1:2048
    V(k,1)=d((k-1)/2048);
end
x=1:2048;
m=11;
% Set periodization extension mode
dwtmode('per');
% 0-10 levels
for j=1:m-1
    % Apply D4QWT in Figure 6.12 on the vector V
    V1=F2NK(m,j-1)*V;
    % Apply classical D4 wavelet transform on the vector V
    [C2,L2] = wavedec(V,j,'db2');
    %The comparisons of D4QWT and classical D4 wavelet
transform
    err1(j,1) = norm(V1-C2);
    % Apply the inverse of D4QWT in Figure 6.13 on the vector
V1
    V2=IF2NK(m,j-1)*V1;
    %The comparisons of V2 and V
    err2(j,1) = norm(V2-V);
end
end
```

## The Codes for Quantum Wavelet Packet Transforms in Chapter 7

### Code B.58 The code for 1D-HQWPT in Figure 7.3

```
% k>=0 and k<n
function R=ODHQWPT(n,k)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0;0 1];
if k==n-1
    % 1-dimensional Haar quantum wavelet packet transform in
Figure 7.3 (c)
    R=allpermutation(n)*tenProduct(H,n);
elseif k==n-2
```

```

    % 1-dimensional Haar quantum wavelet packet transform in
    Figure 7.3 (b)
    R=allpermutation(n)*kron(I,tenProduct(H,n-1));
else
    % 1-dimensional Haar quantum wavelet packet transform in
    Figure 7.3 (a)
    R=allpermutation(n)*kron(allpermutation(n-k-
    1),tenProduct(H,k+1));
end
end

```

**Code B.59** The code for the inverse of 1D-HQWPT in Figure 7.4

```

% k>=0 and k<n
function R=IODHQWPT(n,k)
H=sqrt(2)*0.5*[1 1; 1 -1];
I=[1 0;0 1];
if k==n-1
    % The inverse of 1-dimensional Haar quantum wavelet packet
    transform in Figure 7.4 (c)
    R=tenProduct(H,n)*allpermutation(n);
elseif k==n-2
    % The inverse of 1-dimensional Haar quantum wavelet packet
    transform in Figure 7.4 (b)
    R=kron(I,tenProduct(H,n-1))*allpermutation(n);
else
    % The inverse of 1-dimensional Haar quantum wavelet packet
    transform in Figure 7.4 (a)
    R=kron(allpermutation(n-k-1),tenProduct(H,k+1))*allpermuta
    tion(n);
end
end

```

**Code B.60** The code for 1D-D4QWPT in Figure 7.6

```

% k>=0 and k<n-1
function R=OD4QWPT(n,k)
I=[1 0;0 1];
if k==n-2
    % 1-dimensional D4 quantum wavelet packet transform in
    Figure 7.6 (b)
    R=allpermutation(n);
    for j=2:n-1
        % Call the function D2NP(), which implements the
        circuit in Figure 6.11 (a)
        R=R*kron(D2NP(j),tenProduct(I,n-j));
    end
end

```

## Appendix 2

```
R=R*D2NP(n);
else
    % 1-dimensional Haar quantum wavelet packet transform in
    Figure 7.6 (a)
    R=D2NP(n);
    for j=1:k
        R=kron(D2NP(n-j),tenProduct(I,j))*R;
    end
    R=allpermutation(n)*kron(allpermutation(n-k-
1),tenProduct(I,k+1))*R;
end
end
```

### Code B.61 The code for the inverse of 1D-D4QWPT in Figure 7.6

```
% k>=0 and k<n-1
function R=IOD4QWPT(n,k)
I=[1 0;0 1];
if k==n-2
    % The invert of 1-dimensional D4 quantum wavelet packet
    transform in Figure 7.6 (d)
    R=allpermutation(n);
    for j=2:n-1
        % Call the function ID2NP(), which implements the
        circuit in Figure 6.11 (b)
        R=kron(ID2NP(j),tenProduct(I,n-j))*R;
    end
    R=ID2NP(n)*R;
else
    % The invert of 1-dimensional Haar quantum wavelet packet
    transform in Figure 7.6 (c)
    R=ID2NP(n);
    for j=1:k
        R=R*kron(ID2NP(n-j),tenProduct(I,j));
    end
    R=R*kron(allpermutation(n-k-1),tenProduct(I,k+1))*allpermut
ation(n);
end
end
```

### Code B.62 The code for the first 3 levels of 1D-HQWPT with the input V in (6.46)

```
function []=testFirst3LevelHaarPacket()
clc; clear all
for k=1:2048
    V(1,k)=d((k-1)/2048);
```

```

end
x=1:2048;
m=11;
V1=V';
% 0 level
subplot(4,8,1:8);
plot(x,V1',' .');
% 1 level
subplot(4,8,9:12);
V1=ODHQWPT(m,0)*V';
x=1:1024;
plot(x,V1(x,1)', ' .');
subplot(4,8,13:16)
x=1025:2048;
plot(x,V1(x,1)', ' .');
% 2 levels
V1=ODHQWPT(m,1)*V';
subplot(4,8,17:18);
x=1:512;
plot(x,V1(x,1)', ' .');
subplot(4,8,19:20);
x=513:1024;
plot(x,V1(x,1)', ' .');
subplot(4,8,21:24);
x=1025:2048;
plot(x,V1(x,1)', ' .');
% 3 levels
V1=ODHQWPT(m,2)*V';
subplot(4,8,25);
x=1:256;
plot(x,V1(x,1)', ' .');
subplot(4,8,26);
x=257:512;
plot(x,V1(x,1)', ' .');
subplot(4,8,27:28);
x=513:1024;
plot(x,V1(x,1)', ' .');
subplot(4,8,29:32);
x=1025:2048;
plot(x,V1(x,1)', ' .');
end

```

**Code B.63** The code for the comparisons of 1D-HQWPT, its inverse, and classical Haar wavelet packet transform

```

function []=testComparisonHaarPacket()
clc
clear all

```

## Appendix 2

```
for k=1:2048
    V(1,k)=d((k-1)/2048);
end
x=1:2048;
m=11;
% 0-11 levels
for j=1:m
    % Apply 1D-HQWPT in Figure 7.3 on the vector V'
    V1=ODHQWPT(m,j-1)*V';
    % Apply classical 1D Haar wavelet packet transform on the
    vector V'
    wt = wpdec(V',j,'dbl');
    C2=[];
    for i=0: 2^j-1
        cfs=wpcoef(wt,[j,i]);
        C2=[C2; cfs];
    end
    %The comparison of 1D-HQWPT and classical Haar wavelet
    packet transform
    err1(j,1) = norm(V1-C2);
    % Apply the inverse of 1D-HQWPT in Figure 7.4 on the vector
    V1
    V2=IODHQWPT(m,j-1)*V1;
    %The comparisons of V2 and V'
    err2(j,1) = norm(V2-V');
end
end
```

**Code B.64** The code for the first 3 levels of 1D-D4QWPT with the input V in (6.46)

```
function []=testFirst3LevelD4Packet()
clc
clear all
for k=1:2048
    V(1,k)=d((k-1)/2048);
end
x=1:2048;
m=11;
V1=V';
% 0 level
subplot(4,8,1:8);
plot(x,V1','.'');
% 1 level
subplot(4,8,9:12);
V1=OD4QWPT(m,0)*V';
x=1:1024;
plot(x,V1(x,1)','.'');
```

```

subplot(4,8,13:16)
x=1025:2048;
plot(x,V1(x,1)',','.');
% 2 levels
V1=OD4QWPT(m,1)*V';
subplot(4,8,17:18);
x=1:512;
plot(x,V1(x,1)',','.');
subplot(4,8,19:20);
x=513:1024;
plot(x,V1(x,1)',','.');
subplot(4,8,21:24);
x=1025:2048;
plot(x,V1(x,1)',','.');
% 3 levels
V1=OD4QWPT(m,2)*V';
subplot(4,8,25);
x=1:256;
plot(x,V1(x,1)',','.');
subplot(4,8,26);
x=257:512;
plot(x,V1(x,1)',','.');
subplot(4,8,27:28);
x=513:1024;
plot(x,V1(x,1)',','.');
subplot(4,8,29:32);
x=1025:2048;
plot(x,V1(x,1)',','.');
end

```

**Code B.65** The code for the comparisons of 1D-D4QWPT, its inverse, and classical D4 wavelet packet transform

```

function []=testComparisonHaarPacket()
clc
clear all
for k=1:2048
    V(1,k)=d((k-1)/2048);
end
x=1:2048;
m=11;
% Set periodization extension mode
dwtmode('per');
% 0-10 levels
for j=1:m-1
    % Apply 1D-D4QWPT in Figure 7.5 on the vector V'
    V1=OD4QWPT(m,j-1)*V';
    % Apply classical D4 wavelet packet transform on the

```

## Appendix 2

```
vector V'
    wt = wpdec(V',j,'db2');
    C2=[];
    for i=0: 2^j-1
        cfs=wpcoef(wt,[j,i]);
        C2=[C2; cfs];
    end
    % The comparison of 1D-D4QWPT and classical D4 wavelet
transform
    err1(j,1) = norm(V1-C2);
    % Apply the inverse of 1D-D4QWPT in Figure 7.5 on the
vector V1
    V2=IOD4QWPT(m,j-1)*V1;
    % The comparison of V2 and V'
    err2(j,1) = norm(V2-V');
end
end
```

**Code B.66** The code for 2D-HQWPT in Figure 7.7 and Figure 7.78

```
% h>=0 and h<min(m,k)
function R=TDHQWPT(m,k,h)
R=kron(ODHQWPT(m,h),ODHQWPT(k,h));
end
```

**Code B.67** The code for the inverse of 2D-HQWPT in Figure 7.7 and Figure 7.8

```
% h>=0 and h<min(m,k)
function R=ITDHQWPT(m,k,h)
R=kron(IODHQWPT(m,h),IODHQWPT(k,h));
end
```

**Code B.68** The code for 2D-D4QWPT in Figure 7.7 and Figure 7.78

```
% l>=0 and l<min(m,k)-1
function R=TD4QWPT(m,k,l)
R=kron(OD4QWPT(m,l),OD4QWPT(k,l));
end
```

**Code B.69** The code for the inverse of 2D-D4QWPsT in Figure 7.7 and Figure 7.8

```
% l>=0 and l<min(m,k)-1
function R=ITD4QWPT(m,k,l)
R=kron(IOD4QWPT(m,l),IOD4QWPT(k,l));
end
```

**Code B.70** The code for the first 2 levels of 2D-HQWPT with a 256\*256 image

```

function []=testFirst2Level2DHaarPacket()
clear all
clc
% f is a 256*256 image
f = double(imread('Cameraman.bmp'));
subplot(1,3,1);
imshow(uint8(f),[]); %The original image
[row,col]=size(f);
m=log2(row);
k=log2(col);
% Call the fuction GQ2 to create a NASS state
[G,Q]=GQ2(f);
% Apply 1-level 2D-HQWPT on the image
QH1=TDHQWPT(m,k,0)*Q;
% Retrieve the transformed image by 1-level 2D-HQWPT
QH1I=GQ2(QH1, G, row, col);
subplot(1,3,2),imshow(uint8(QH1I));
% Apply 2-level 2D-HQWPT on the image
QH2=TDHQWPT(m,k,1)*Q;
% Retrieve the transformed image by 2-level 2D-HQWPT
QH2I=GQ2(QH2, G, row, col);
subplot(1,3,3),imshow(uint8(QH2I));
end

```

**Code B.71** The code for the synthesized matrix by wavelet packet coefficients

```

% A consists of wavelet packet coefficients of the j-level
wavelet packet transform
function B=Com(A,j)
if j > 0
    % The multi-level wavelet packet transform
    [x,y]=size(A);
    z=y/4;
    A1=A(:,1:z);
    A2=A(:,z+1:2*z);
    A3=A(:,2*z+1:3*z);
    A4=A(:,3*z+1:4*z);
    u1=Com(A1,j-1);
    u2=Com(A2,j-1);
    u3=Com(A3,j-1);
    u4=Com(A4,j-1);
    B=[u1 u3; u2 u4];
else
    % The single-level wavelet packet transform
    B=A;
    return;
end

```

## Appendix 2

```
end  
end
```

**Code B.72** The code for the comparisons of 2D-HQWPT, its inverse, and classical 2D Haar wavelet packet transform

```
function []=testComparison2DHaarPacket()  
clear all  
clc  
% Ag is a 256*256 image  
Ag = double(imread('Cameraman.bmp'));  
[row,col]=size(Ag);  
m=log2(row);  
k=log2(col);  
% Call the function GQ2 to create a NASS state  
[G,Q]=GQ2(Ag);  
h=min(m,k);  
% 0-7 levels  
for j=1:h  
    % Apply 2D-HQWPT on the quantum image Q  
    QH=TDHQWPT(m,k,j-1)*Q;  
    % Retrieve the transformed image  
    QHI=QG2(QH, G, row, col);  
    % Apply classical 2D Haar wavelet packet transform on the  
    classical image Ag  
    wpt = wpdec2(Ag,j,'db1');  
    C2=[];  
    for i=0: 4^j-1  
        cfs=wpcoef(wpt,[j,i]);  
        C2=[C2 cfs];  
    end  
    % Call the function Com() to create a matrix by using  
    coefficients of  
    % (j+1)-level wavelet packet transform  
    mdwt2=Com(C2,j);  
    %The comparison of 2D-HQWPT and classical 2D Haar wavelet  
    packet transform  
    err1(j,1) = norm(QHI-mdwt2);  
    % Apply the inverse of 2D-HQWPT on the quantum image QH  
    QH2=ITDHQWPT(m,k,j-1)*QH;  
    % Retrieve the transformed image by the inverse of 2D-HQWPT  
    QH2I=QG2(QH2, G, row, col);  
    %The comparisons of QH2I and Ag  
    err2(j,1) = norm(QH2I-Ag);  
end  
end
```

**Code B.73** The code for the first 2 levels of 2D-D4QWPT with a 256\*256 image

```
function []=testFirst2Level2DD4Packet()
clear all
clc
% f is a 256*256 image
f = double(imread('Cameraman.bmp'));
subplot(1,3,1);
imshow(uint8(f),[]); %The original image
[row,col]=size(f);
m=log2(row);
k=log2(col);
% Call the fuction GQ2 to create a NASS state
[G,Q]=GQ2(f);
% Apply 1-level 2D-D4QWPT on the image
QD1=TD4QWPT(m,k,0)*Q;
% Retrieve the transformed image by 1-level 2D-D4QWPT
QD1I=QG2(QD1, G, row, col);
subplot(1,3,2),imshow(uint8(QD1I));
% Apply 2-level 2D-D4QWPT on the image
QD2=TD4QWPT(m,k,1)*Q;
% Retrieve the transformed image by 2-level 2D-D4QWPT
QD2I=QG2(QD2, G, row, col);
subplot(1,3,3),imshow(uint8(QD2I));
end
```

**Code B.74** The code for the comparisons of 2D-D4QWPT, its inverse, and classical 2D D4 wavelet packet transform

```
function []=testComparison2DD4Packet()
clear all
clc
% Ag is a 256*256 image
Ag = double(imread('Cameraman.bmp'));
[row,col]=size(Ag);
m=log2(row);
k=log2(col);
% Call the function GQ2 to create a NASS state
[G,Q]=GQ2(Ag);
h=min(m,k)-1;
% Set periodization extension mode
dwtmode('per');
% 0-6 levels
for j=1:h
    % Apply 2D-D4QWPT on the quantum image Q
    QD=TD4QWPT(m,k,j-1)*Q;
```

## Appendix 2

```
% Retrieve the transformed image
QDI=QG2(QD, G, row, col);
% Apply classical 2D D4 wavelet packet transform on the
classical image Ag
wpt = wpdec2(Ag,j,'db2');
C2=[];
for i=0: 4^j-1
    cfs=wpcoef(wpt,[j,i]);
    C2=[C2 cfs];
end
% Call the function Com() to create a matrix by using
coefficients of
% j-level wavelet packet transform
mdwt2=Com(C2,j);
%The comparison of 2D-D4QWPT and classical 2D Haar wavelet
packet transform
err1(j,1) = norm(QDI-mdwt2);
% Apply the inverse of 2D-D4QWPT on the quantum image QD
QD2=ITD4QWPT(m,k,j-1)*QD;
% Retrieve the transformed image by the inverse of
2D-D4QWPT
QD2I=QG2(QD2, G, row, col);
%The comparisons of QD2I and Ag
err2(j,1) = norm(QD2I-Ag);
end
end
```

**Code B.75** The code for 3D-HQWPT in Figure 7.9 and Figure 7.10

```
% i>=0 and i<min(m,k,h)
function R=ThDHQWPT(m,k,h,i)
R=kron(kron(ODHQWPT(m,i),ODHQWPT(k,i)),ODHQWPT(h,i));
end
```

**Code B.76** The code for the inverse of 3D-HQWPT in Figure 7.9 and Figure 7.10

```
% i>=0 and i<min(m,k,h)
function R=IThDHQWPT(m,k,h,i)
R=kron(kron(IODHQWPT(m,i),IODHQWPT(k,i)),IODHQWPT(h,i));
end
```

**Code B.77** The code for the first 2 levels of 3D-HQWPT for a 64\*32\*8 video

```
function []=testFirst2Level3DHaarPacket()
clear all
```

```

clc
% Vg is a 64*32*8 video
for j=1:8
    A=imread([strcat('v',num2str(j)),'.BMP']);
    Vg(:,:,j)=double(A);
    subplot(3,8,j);
    imshow(A,[]);
end
[row, col,st]=size(Vg);
m=log2(row);
k=log2(col);
h=log2(st);
% Call the fuction GQ3 to create a NASS state
[G,Q]=GQ3(Vg);
% Apply 1-level 3D-HQWPT on the video
QH1=ThDHQWPT(m,k,h,0)*Q;
% Retrieve the transformed image by 1-level 3D-HQWPT
QH1I=QG3(QH1, G, row, col, st);
for j=1:8
    A=QH1I(:,:,j);
    subplot(3,8,8+j);
    imshow(A,[]);
end
% Apply 2-level 3D-HQWPT on the video
QH2=ThDHQWPT(m,k,h,1)*Q;
% Retrieve the transformed image by 2-level 3D-HQWPT
QH2I=QG3(QH2, G, row, col, st);
for j=1:8
    A=QH2I(:,:,j);
    subplot(3,8,16+j);
    imshow(A,[]);
end
end
end

```

### **Code B.78** The code for classical 3D $j$ -level wavelet packet transform in (7.24)

```

% X is a video;
% wname is a wavelet name;
function B=WPT3(X,j,wname)
[row, col,st]=size(X);
% Applying 2D  $j$ -level wavelet packet transform on image frames
of the video X
for k=1:st
    wpt = wpdec2(X(:,:,k),j,wname);
    C2=[];
    for i=0: 4^j-1
        cfs=wpcoef(wpt,[j,i]);
        C2=[C2 cfs];
    end
end

```

## Appendix 2

```
end
% Call the function Com() to create a matrix by using
coefficients of
% j-level wavelet packet transform
mwpt2(:, :, k) = Com(C2, j);
end
% Applying the third dimensional wavelet packet transform on
the video mwpt2
for i1=1:row
    for i2=1:col
        V1=[];
        for i3=1:st
            tt=mwpt2(:, :, i3);
            V1=[V1; tt(i1, i2)];
        end
        wt = wpdec(V1, j, wname);
        C2=[];
        for i=0: 2^j-1
            cfs=wpcoef(wt, [j, i]);
            C2=[C2; cfs];
        end
        B(i1, i2, :) = C2;
    end
end
end
end
```

**Code B.79** The code for the comparisons of 3D-HQWPT, its inverse, and classical 3D Haar wavelet packet transform

```
function []=testComparison3DHaarPacket()
clear all
clc
% Vg is a 64*32*8 video
for j=1:8
    A=imread([strcat('v', num2str(j)), '.BMP']);
    Vg(:, :, j)=double(A);
end
[row, col, st]=size(Vg);
m=log2(row);
k=log2(col);
h=log2(st);
% Call the function GQ3 to create a NASS state
[G, Q]=GQ3(Vg);
x=min(min(m, k), h);
% 0-2 levels
for j=1:x
    % Apply 3D-HQWPT on the quantum image Q
    QH=ThDHQWPT(m, k, h, j-1)*Q;
```

```

    % Retrieve the transformed image
    QHI=QG3(QH, G, row, col,st);
    % Apply classical 3D Haar wavelet packet transform on the
video Vg
    B= WPT3(Vg,j,'db1');
    %The comparison of 3D-HQWPT and classical 3D Haar wavelet
packet transform
    sum=0;
    for i3=1:st
        sum=sum+(norm(QHI(:,:,i3)-B(:,:,i3)))^2;
    end
    err1(j,1)=sqrt(sum);
    % Apply the inverse of 3D-HQWPT on the quantum image QH
    QH2=IThDHQWPT(m,k,h,j-1)*QH;
    % Retrieve the transformed image by the inverse of 3D-HQWPT
    QH2I=QG3(QH2, G, row, col, st);
    %The comparisons of QH2I and Vg
    sum=0;
    for i3=1:st
        sum=sum+(norm(QH2I(:,:,i3)-Vg(:,:,i3)))^2;
    end
    err2(j,1)=sqrt(sum);
end
end

```

**Code B.80** The code for the first 2 levels of 3D-D4QWPT with a 64\*32\*8 video

```

function []=testFirst2Level3DD4Packet()
clear all
clc
% Vg is a 64*32*8 video
for j=1:8
    A=imread([strcat('v',num2str(j)),'.BMP']);
    Vg(:,:,j)=double(A);
    subplot(3,8,j);
    imshow(A,[]);
end
[row, col,st]=size(Vg);
m=log2(row);
k=log2(col);
h=log2(st);
% Call the fuction GQ3 to create a NASS state
[G,Q]=GQ3(Vg);
% Apply 1-level 3D-D4QWPT on the video
QD1=ThD4QWPT(m,k,h,0)*Q;
% Retrieve the transformed image by 1-level 3D-D4QWPT
QD1I=QG3(QD1, G, row, col,st);

```

## Appendix 2

```
for j=1:8
    A=QD1I(:,:,j);
    subplot(3,8,8+j);
    imshow(A, []);
end
% Apply 2-level 3D-D4QWPT on the video
QD2=ThD4QWPT(m,k,h,1)*Q;
% Retrieve the transformed image by 2-level 3D-D4QWPT
QD2I=QG3(QD2, G, row, col,st);
for j=1:8
    A=QD2I(:,:,j);
    subplot(3,8,16+j);
    imshow(A, []);
end
end
```

**Code B.81** The code for the comparisons of 3D-D4QWPT, its inverse, and classical 3D D4 wavelet packet transform

```
function []=testComparison3DD4Packet()
clear all
clc
% Vg is a 64*32*8 video
for j=1:8
    A=imread([strcat('v',num2str(j)),'.BMP']);
    Vg(:,:,j)=double(A);
end
[row, col,st]=size(Vg);
m=log2(row);
k=log2(col);
h=log2(st);
% Set periodization extension mode
dwtmode('per');
% Call the function GQ3 to create a NASS state
[G,Q]=GQ3(Vg);
x=min(min(m,k),h)-1;
% 0-1 levels
for j=1:x
    % Apply 3D-D4QWPT on the quantum image Q
    QD=ThD4QWPT(m,k,h,j-1)*Q;
    % Retrieve the transformed image
    QDI=QG3(QD, G, row, col,st);
    % Apply classical 3D D4 wavelet packet transform on the
    video Vg
    B= WPT3(Vg,j,'db2');
    %The comparison of 3D-D4QWPT and classical 3D D4 wavelet
    packet transform
    sum=0;
```

```

    for i3=1:st
        sum=sum+(norm(QDI(:,:,i3)-B(:,:,i3)))^2;
    end
    err1(j,1)=sqrt(sum);
    % Apply the inverse of 3D-D4QWPT on the quantum image QD
    QD2=IThD4QWPT(m,k,h,j-1)*QD;
    % Retrieve the transformed image by the inverse of
3D-D4QWPT
    QD2I=QG3(QD2, G, row, col, st);
    %The comparisons of QD2I and Vg
    sum=0;
    for i3=1:st
        sum=sum+(norm(QD2I(:,:,i3)-Vg(:,:,i3)))^2;
    end
    err2(j,1)=sqrt(sum);
end
end

```

**Code B.82** The code for quantum image compression based on h-level 2D-QWPTs

```

% A is a row*col grayscale images;
% Type is equal to 0 or 1, which denotes Haar wavelet or D4
wavelet
function [QHCI,Qr,psnr1]=QICQWPT(A,h,Type)
[ row,col]=size(A);
m=log2(row);
k=log2(col);
% Call the function GQ2 to create four NASS states
[G,Q]=GQ2(A);
if Type==0
    % Apply h-level 2D-HQWPT on the image
    QH=TDHQWPT(m,k,h-1)*Q;
else
    % Apply h-level 2D-D4QWPT on the image
    QH=TD4QWPT(m,k,h-1)*Q;
end
% Get the threshold value
a=1.2*mean(abs(QH(:)));
% Retrieve the compressed vectors from QH
QH(abs(QH)<a)=0;
% Calculate quantum compression ratio
N=row*col;
B=(QH~=0);
Qr=N/sum(B);
% Retrieve the compressed images
if Type==0
    QHC=ITDHQWPT(m,k,h-1)*QH; % Apply the inverse of h-level

```

## Appendix 2

```
2D_HQWPT on QH
else
    QHC=ITD4QWPT(m,k,h-1)*QH; % Apply the inverse of h-level
2D_HQWPT on QH
end
QHCI=QG2(QHC, G, row, col);
% Call the function psnr() to calculate PSNR
psnr1=psnr(A,QHCI,max(max(A)));
end
```

### Code B.83 The code for the examples of quantum image compression based on 3-level 2D-QWPTs

```
function []=testQICQWPT()
clear all; clc
% A stores four 256*256 grayscale images
A(:, :, 1)= double(imread('Cameraman.bmp'));
A(:, :, 2)= double(imread('glena.bmp'));
A(:, :, 3)= double(imread('gbaboon.bmp'));
A(:, :, 4)= double(imread('gairplane.bmp'));
for j=1:4
    subplot(3,4,j);
    imshow(uint8(A(:, :, j)), []);
end
% Call the function QICQWPT to realize quantum image
compression for 3-level 2D-HQWPT
Qrh=zeros(4,1);
Psnrh=zeros(4,1);
for j=1:4
    [QHCI,Qr,psnr1]=QICQWPT(A(:, :, j),3,0);
    Qrh(j,1)=Qr; % Get quantum compression ratio
    Psnrh(j,1)=psnr1; % Get peak signal-to-noise ratio
    subplot(3,4,4+j);
    imshow(uint8(QHCI), []);
end
Qrhm=mean(Qrh); % Get the mean value of QRCs for 3-level
2D-HQWPT
Psnrhm=mean(Psnrh); % Get the mean value of PNSRs for 3-level
2D-HQWPT
% Call the function QICQWPT to realize quantum image
compression for 2D-D4QWPT
dwtmode('per'); % Set periodization extension mode
Qrd=zeros(4,1);
Psnrd=zeros(4,1);
for j=1:4
    [QHCI,Qr,psnr1]=QICQWPT(A(:, :, j),3,1);
    Qrd(j,1)=Qr;
    Psnrd(j,1)=psnr1;
```

```

        subplot(3,4,8+j);
        imshow(uint8(QHCI),[]);
    end
    Qrdm=mean(Qrd); % Get the mean value of QRCs for 3-level
    2D-D4QWPT
    Psnrdm=mean(Psnrd); % Get the mean value of PNSRs for 3-level
    2D-D4QWPT
end

```

## The Codes for Quantum Image Segmentation in Chapter 8

### Code B.84 The code for quantum Hadamard edge detection algorithm

```

% Ag is a 2^n*2^n binary image
function OutputI=QHED(Ag)
[ row,col]=size(Ag);
    n=log2(row)+log2(col);
    H=sqrt(2)*0.5*[1 1; 1 -1];
    I=[1 0;0 1];
    % Encode the image Ag into a quantum state Q;
    fg=double(Ag);
    dg=abs(fg).*abs(fg);
    G=sqrt(sum(dg(:)));
    g = fg/G; % Normalize the image
    Q=d2tod1(g); %Convert the image to a column vector
    % Create the input of the algorithm QHED
    Qin=kron(Q,[1;0]);
    % Apply the circuit in Figure 8.3 on the input Qin
    R=kron(tenProduct(I,n),H)*Qin;
    R=Q2N(n+1)*R;
    R=kron(tenProduct(I,n),H)*R;
    % Denormalize the vector R
    DR=R*G;
    % Read the values of the even positions of the vector DR
    Qout=DR(2:2:end);
    % Obtain the output image
    OutputI=dltod2(2*Qout, row, col);
end

```

### Code B.85 The code for the example of quantum Hadamard edge detection algorithm

```

function []=testQHED()
clear all
clc

```

## Appendix 2

```
% Ag is a 128*128 binary image
Ag = imread('Cat.bmp');
subplot(1,2,1);
imshow(Ag); %The original image
% Call QHED function to perform quantum Hadamard edge detection
algorithm
OutputI=QHED(Ag);
% Display the edge of the image Ag
OutputI2=abs(OutputI);
OutputB=im2bw(OutputI2); % Converts the image OutputI2 to a
binary image
subplot(1,2,2);
imshow(OutputB);
end
```

### Code B.86 The implementation of generalized Grover search algorithm in Figure 8.10

```
% A is the set of marked states
function Gph=GGrover(n,A)
I=[1 0;0 1];
X=[0 1; 1 0];
X1=-X;
H=sqrt(2)*0.5*[1 1; 1 -1];
[m1,m2]=size(A);
% The implementation of the operator U in Figure 8.8 (a)
U=kron(tenProduct(H,n),I);
for j=m1:-1:1
    U=Cup(n,A(j),X)*U;
end
% The implementation of the inverse of the operator U
IU=U';
% The implementation of R0 in Figure 8.9 (a)
R0=kron(tenProduct(I,n-1),X1*H)*Cup(n-1,0,X)*kron(tenProduct(I,n-1),H*X);
% The implementation of R1 in Figure 8.9 (b)
R1=kron(tenProduct(I,n),H*X*H);
% The implementation of generalized Grover search algorithm in
Figure 8.10
Gph=U*kron(R0,I)*IU*R1;
end
```

**Code B.87** The code for the first example of quantum image segmentation based on generalized Grover search algorithm

```
function []=testQISGGSA1()
clear all
clc
I=[1 0;0 1];
X=[0 1; 1 0];
State0=[1;0];
% S is a 135*1 vector which stores the positions of the image
edge
S= [964;965;966;967;968;1028;1033;1034;1092;1099;1156;1164;1221
;1229;1286;
    1294;1350;1359;1414;1424;1478;1489;1543;1554;1607;1619;1620
;1640;1641;
    1642;1643;1644;1645;1646;1647;1648;1649;1671;1685;1686;1687
;1688;1691;
    1692;1693;1695;1696;1697;1698;1699;1700;1701;1702;1703;1714
;1715;1735;
    1753;1754;1758;1780;1781;1798;1846;1847;1848;1849;1860;1861
;1914;1915;
    1916;1924;1981;1982;1983;1984;1988;2031;2032;2033;2034;2035
;2036;2037;
    2038;2039;2040;2041;2042;2043;2044;2053;2054;2055;2094;2120
;2158;2185;
    2186;2187;2188;2189;2190;2191;2199;2215;2216;2217;2218;2219
;2220;2221;
    2256;2257;2262;2264;2265;2267;2268;2269;2270;2271;2272;2273
;2274;2275;
    2276;2277;2278;2322;2323;2324;2325;2330];
[m1,m2]=size(S);
% Ag is a 64*64 grayscale image
Ag = double(imread('airplane64.bmp'));
subplot(1,2,1);
imshow(uint8(Ag));
[row, col]=size(Ag);
m=log2(row);
k=log2(col);
n=m+k;
% Call the function GQ2 to create a NASS state
[G,Q]=GQ2(Ag);
% Create a NAQSS state
Q1=kron(Q,State0);
for j=1:m1
    Q1=Cup(n,S(j),X)*Q1;
end
% Calculate N, r, the averages of the amplitudes, and the
variances of the amplitudes
```

## Appendix 2

```

N=row*col;
r=m1;
A1=Q1(1:2:end);
A2=Q1(2:2:end);
La0=sum(A1)/(N-r); % The average of the amplitudes of unmarked
states
Ka0=sum(A2)/r; % The average of the amplitudes of marked states
% The variance of the amplitudes of unmarked states
Dta10=sum(abs(A1-La0).*abs(A1-La0))/(N-r);
% Calculate the optimal iterations of Grover in (8.46)
ph=atan(Ka0*sqrt(r/(N-r))/La0);
w=acos(1-2*r/N);
t=round((pi()/2-ph)/w);
% Apply the t iterations of generalized Grover search algorithm
on the NAQSS state Q1
Gr=GGrover(n,S);
for j=1:t
    Q1=Gr*Q1;
end
% Calculate the average of the amplitudes of marked states of
Q1
A3=Q1(2:2:end);
Kat=sum(A3)/r;
% Calculate the probability of success
Pt= sum(abs(A3).*abs(A3));
% Retrieve the target sub-image
A3(A3~=0) = A3(A3~=0)-Kat+Ka0;
QI=QG2(A3, G, row, col);
subplot(1,2,2);
imshow(uint8(QI));
end

```

## Code B.88 The implementation of the generalized Grover search algorithm in (8.56)

```

function Gph=GGrover2(n,A) % A is the binary image of a target
sub-image
I=[1 0;0 1];
X=[0 1; 1 0];
t0=[1;0]; % The state |0>
t1=[0;1]; % The state |1>
H=sqrt(2)*0.5*[1 1; 1 -1];
[m1,m2]=size(A);
N=2^n;
B1=double(A);
B2=double(~A);
V1=d2tod1(B1); % Convert the image B1 to a column vector
V2=d2tod1(B2); % Convert the image B2 to a column vector

```

```

Ph1=(kron(V2,t0)+kron(V1,t1))/sqrt(N); % Calculate the state in
(8.29)
Ph2=(kron(V2,t1)+kron(V1,t0))/sqrt(N); % Calculate the state in
(8.54)
R1=kron(tenProduct(I,n),H*X*H); % Calculate R1 in Figure 8.9
(b)
% The implementation of generalized Grover search algorithm in
(8.56)
Gph=(2*Ph1*Ph1'+2*Ph2*Ph2'-tenProduct(I,n+1))*R1;
end

```

**Code B.89** The code for the second example of quantum image segmentation based on the generalized Grover search algorithm

```

function []=testQISGSA2()
clear all
clc
% Ag is a 128*128 color image
Ag = imread('airplane128.bmp');
subplot(1,3,1);
imshow(Ag);
Ag=double(Ag);
[row,col,tem]=size(Ag);
m=log2(row);
k=log2(col);
n=m+k;
% Tar is the 128*128 binary image of a target sub-image
Tar=imread('airplaneBin.bmp');
subplot(1,3,2);
imshow(Tar);
% Call the function CQ2 to create a NASS state Q
[G,Q]=CQ2(Ag);
% Create a NAQSS state Phx
t0=[1;0]; % The state |0>
t1=[0;1]; % The state |1>
B1=double(Tar);
B2=double(~Tar);
V1=d2tod1(B1); % Convert the image B1 to a column vector
V2=d2tod1(B2); % Convert the image B2 to a column vector
Q1=Q.*V1;
Q2=Q.*V2;
Phx=kron(Q2,t0)+kron(Q1,t1);
% Calculate N, r, the averages of the amplitudes, and the
variances of the amplitudes
N=row*col;
r=nnz(Tar); % Number of nonzero elements
A1=Phx(1:2:end);
A2=Phx(2:2:end);

```

## Appendix 2

```
La0=sum(A1)/(N-r);% The average of the amplitudes of unmarked
states
Ka0=sum(A2)/r;% The average of the amplitudes of marked states
% The variance of the amplitudes of unmarked states
Dt10=sum(abs(A1-La0).*abs(A1-La0))/(N-r);
% Calculate the optimal iterations of Grover in (8.46)
ph=atan(Ka0*sqrt(r/(N-r))/La0);
w=acos(1-2*r/N);
t=round((pi()/2-ph)/w);
% Apply the t iterations of generalized Grover search algorithm
on the NAQSS state Phx
Gr=GGrover2(n,Tar);
for j=1:t
    Phx=Gr*Phx;
end
% Calculate the average of the amplitudes of marked states of
Phx
A3=Phx(2:2:end);
Kat=sum(A3)/r;
% Calculate the probability of success
Pt= sum(abs(A3).*abs(A3));
% Retrieve the target sub-image
A3(A3~=0) = A3(A3~=0)-Kat+Ka0;
QI=QC2(A3, G, row, col);
subplot(1,3,3);
imshow(uint8(QI),[]);
end
```